📖 DAY 4: COMPLETE TOPIC-BY-TOPIC STUDY GUIDE

Distributed AI Training, Slurm Multi-Node Clusters & Intel VTune Profiling Optimization

GLA University — Department of Computer Science & Engineering | BCSG701

📑 TABLE OF CONTENTS

1. Module 1: Distributed Computing Foundations on Supercomputers
2. Module 2: Multi-Node Slurm Scheduling & Cluster Orchestration
3. Module 3: Collective Communications & AllReduce Topologies
4. Module 4: Multi-Node Horovod & Distributed Data Parallel (DDP)
5. Module 5: Intel VTune Profiler & Top-down Microarchitecture Analysis (TMAM)
6. Module 6: Intel Advisor & The Roofline Model
7. Module 7: MPI Interconnect Profiling with ITAC
8. Module 8: Real-World Case Study: 4-Node Scaling Optimization

MODULE 1: Distributed Computing Foundations on Supercomputers

1.1 The Need for Distributed AI

In modern high-performance computing, deep learning models have expanded far beyond the memory and computational limits of a single server node. Large Vision Transformers (ViTs), foundation models, and multi-modal neural networks require tens to hundreds of gigabytes of memory just to store parameters, gradients, and optimizer states (Adam m and v vectors).

$$\text{Memory Required (Adam FP32)} = 4 \times \text{params} + 4 \times \text{grads} + 8 \times \text{optimizer states} = 16 \times \text{params (bytes)}$$

A 10-billion parameter model requires over 160 GB of RAM solely for state storage, completely ignoring activation tensors. To train such models in reasonable timeframes, workloads must be distributed across an interconnected cluster of compute nodes.

1.2 Theoretical Scaling Laws: Amdahl vs. Gustafson

Amdahl's Law (Strong Scaling)

Amdahl's Law models **Strong Scaling**: problem size remains fixed while the number of processing processors (N) increases. If s is the serial fraction of the code that cannot be parallelized:

$$S_{\text{latency}}(N) = \frac{1}{(1-p) + \frac{p}{N}} = \frac{1}{s + \frac{1-s}{N}}$$

As $N \to \infty$, the maximum theoretical speedup is strictly capped by $\frac{1}{s}$. If 5% of your training loop is serial (I/O, barrier synchronization, single-node logging), the maximum achievable speedup across an infinite number of nodes is $\frac{1}{0.05} = 20\times$.

Gustafson's Law (Weak Scaling)

Gustafson's Law models **Weak Scaling**: the workload per processor is kept constant as the number of processors increases (total problem size grows proportionally with cluster size).

$$S_{\text{scaled}}(N) = s + N(1 - s) = N - s(N - 1)$$

Weak scaling is the primary design metric for HPC AI clusters: as you add compute nodes, you increase the global batch size or dataset volume, keeping per-node memory utilization constant and achieving near-linear throughput scaling.

```
+-------------------------------------------------------------------------+
|                     STRONG SCALING VS WEAK SCALING                      |
+---------------------+--------------------------+------------------------+
| Feature             | Strong Scaling (Amdahl)  | Weak Scaling (Gustafson)  |
+---------------------+--------------------------+------------------------+
| Total Problem Size  | Fixed                    | Grows linearly with N  |
| Work per Processor  | Decreases (1/N)          | Constant               |
| Primary Bottleneck  | Synchronization & Latency| Interconnect Bandwidth |
| HPC Target Metric   | Time-to-Solution         | Maximum Problem Capacity  |
+-------------------------------------------------------------------------+
```

1.3 Supercomputing Interconnects: InfiniBand & RDMA

Ethernet TCP/IP introduces heavy OS kernel-space context switching, buffer copying, and TCP stack processing overheads (~10–30 µs latency). In contrast, HPC clusters utilize **InfiniBand (IB)** or RoCE (RDMA over Converged Ethernet) with **Remote Direct Memory Access (RDMA)**:

- **Kernel-Bypass**: The network interface card (HCA - Host Channel Adapter) directly transfers bytes between user-space virtual memory buffers across nodes without copying data through the operating system kernel.
- **Latency & Bandwidth**: InfiniBand HDR delivers 200 Gbps at sub-1.0 µs MPI ping-pong latency; NDR delivers 400 Gbps.

MODULE 2: Multi-Node Slurm Scheduling & Cluster Orchestration

2.1 Slurm Architecture

Slurm (Simple Linux Utility for Resource Management) is the de facto workload manager for top supercomputers globally:

1. `slurmctld` (Control Daemon): Runs on the head node, maintaining cluster state, job queues, partition priorities, and resource reservations.
2. `slurmd` (Compute Node Daemon): Runs on each compute worker, monitoring CPU/RAM/GPU utilization, spawning tasks via Linux cgroups, and reporting node health.
3. `slurmdbd` (Database Daemon): Connects to MySQL/MariaDB for accounting, fair-share scheduling, and user bank limits.

```
       [ Client CLI / Sbatch / Salloc ]
                      |
                      ▼
        [ slurmctld (Head Node) ] ◄──► [ slurmdbd (Accounting DB) ]
        ┌─────────────┴─────────────┐
        ▼                           ▼
[ slurmd (Node 01) ]    [ slurmd (Node 02) ]
 ├── Linux Cgroups       ├── Linux Cgroups
 ├── CPU / RAM Core Pin  ├── CPU / RAM Core Pin
 └── PMIx Process Spawn  └── PMIx Process Spawn
```

2.2 Deep Dive: Anatomy of a Multi-Node `sbatch` Script

```
#!/bin/bash
#SBATCH --job-name=dist_ai_4node
#SBATCH --partition=compute_nodes
#SBATCH --nodes=4                   # Total physical compute servers
#SBATCH --ntasks-per-node=2         # 2 MPI processes per server (e.g. 1 per socket)
#SBATCH --cpus-per-task=16          # 16 OpenMP/MKL worker threads per MPI process
#SBATCH --time=02:00:00             # Walltime limit HH:MM:SS
#SBATCH --output=logs/job_%j.out    # Standard output log (%j = Job ID)
#SBATCH --error=logs/job_%j.err     # Standard error log

# Load Intel oneAPI Environment
source /opt/intel/oneapi/setvars.sh --force > /dev/null 2>&1

# Threading & Memory Optimization
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OMP_PLACES=cores
export OMP_PROC_BIND=close
export I_MPI_PIN_DOMAIN=auto
export I_MPI_FABRICS=shm:ofi        # Shared memory intradomain, libfabric RDMA interdomain

# Launch distributed job using MPI Process Manager
mpirun -np $SLURM_NTASKS -ppn $SLURM_NTASKS_PER_NODE python train_distributed.py
```

MODULE 3: Collective Communications & AllReduce Topologies

3.1 Collective Operations: Broadcast, Gather, Reduce, AllReduce

In distributed data-parallel training, every worker evaluates forward and backward passes on its local mini-batch. Before optimizer step updates, all workers must average their gradients:

$$\mathbf{G}_{\text{global}} = \frac{1}{P} \sum_{i=1}^{P} \mathbf{G}_i$$

The collective operation required is **AllReduce** (an element-wise Reduce followed by a Broadcast).

3.2 Parameter Server vs. Ring-AllReduce vs. Tree-AllReduce

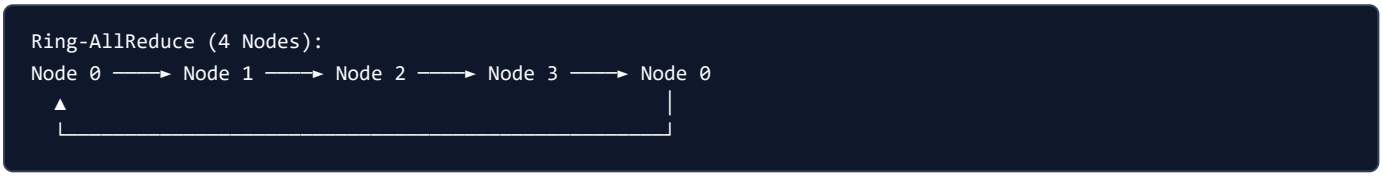
1. **Parameter Server Architecture**:
    - Central parameter server nodes receive gradients from all workers, compute the average, and push updated weights back.

- **Bottleneck**: Network bandwidth of the parameter server scales linearly with the number of workers P ($O(P)$ network traffic at the central node). Severe network bottleneck at > 8 nodes.

2. **Baidu / Horovod Ring-AllReduce**:
   - P processors are arranged in a logical ring.
   - Each process only sends data to its logical successor and receives from its logical predecessor.
   - Executed in two phases: **Scatter-Reduce** followed by **AllGather**.
   - Each buffer of size N is partitioned into P chunks.
   - Total data transferred by each node:

$$\text{Total Transferred} = 2 \times \left(\frac{P-1}{P} \right) N$$

   - **Remarkable Property**: As P grows large, $\frac{P-1}{P} \to 1$. The communication volume per node is completely independent of the number of nodes P!



3. **Hierarchical / Tree-AllReduce (oneCCL & NCCL)**:
   - Within a physical node: Blazing-fast shared memory (SHM) or PCIe/UPI reduces gradients to a master socket rank.
   - Across nodes: InfiniBand RDMA executes Tree or Ring reduction across master ranks only.
   - Greatly minimizes cross-node network latency on multi-thousand node clusters.

MODULE 4: Multi-Node Horovod & Distributed Data Parallel (DDP)

4.1 Horovod Architecture

Horovod wraps PyTorch, TensorFlow, and Apache MXNet with standard MPI communication semantics.

- `hvd.init()`: Initializes MPI world rank, local rank, and communicator size.
- `hvd.DistributedOptimizer()`: Wraps standard optimizers (SGD, Adam) to hook into backward passes and asynchronously execute `allreduce` across workers.
- `hvd.broadcast_variables()`: Synchronizes initial random weights from Rank 0 to all other ranks at Step 0, guaranteeing identical model initialization.

4.2 Local Rank vs. Global Rank

- **Global Rank (`hvd.rank()`)**: Unique ID across the entire cluster ($0 \leq \text{rank} < P$). Rank 0 is designated as the coordinator for checkpoint saving, metric logging, and TensorBoard streaming.
- **Local Rank (`hvd.local_rank()`)**: ID on the local physical machine ($0 \leq \text{local_rank} < \text{devices_per_node}$). Used to bind the process to a specific socket, NUMA node, or GPU.

4.3 Tensor Fusion Buffer Tuning

Horovod utilizes a background coordinator thread and a **Tensor Fusion Ring Buffer** (`HOROVOD_FUSION_THRESHOLD`):

- Instead of triggering thousands of separate tiny network AllReduce calls for individual layer weights (which creates severe latency stalls), Horovod fuses multiple small gradient tensors into a single contiguous buffer (default: 64MB) before initiating the network transfer.
- Tuning `HOROVOD_FUSION_THRESHOLD=67108864` (64 MB) strikes the ideal balance between latency pipelining and memory footprint.

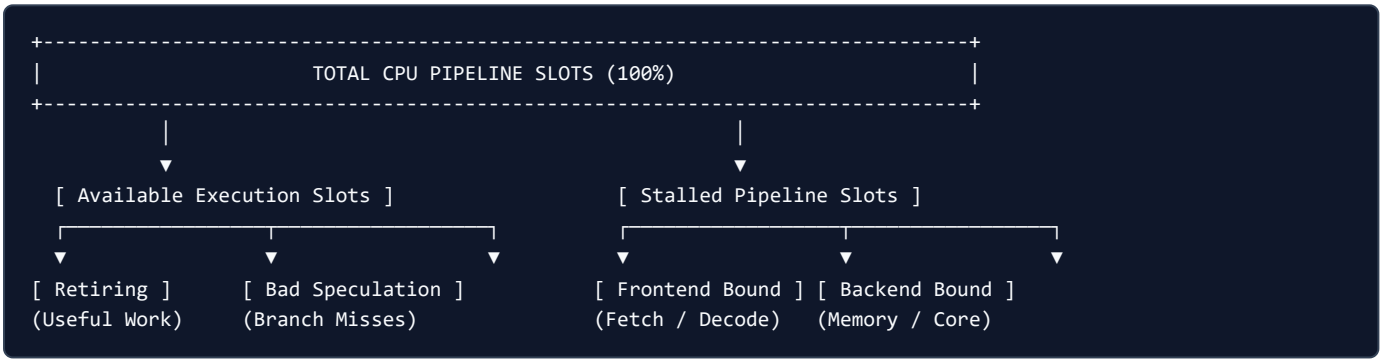
MODULE 5: Intel VTune Profiler & Top-down Microarchitecture Analysis (TMAM)

5.1 Intel VTune Profiler Overview

Intel VTune Profiler is an enterprise performance analysis tool that reads on-chip Hardware Performance Monitoring Units (PMUs) inside Intel Xeon processors to measure execution efficiency without high software instrumentation overhead.

5.2 The TMAM Classification Tree

Developed by Intel architect Ahmad Yasin, the Top-down Microarchitecture Analysis Method classifies all CPU execution pipeline slots into four fundamental buckets:



1. **Retiring (> 40% Target)**:
   - Pipeline slots where micro-operations (μops) successfully executed and retired.
   - High retiring percentage indicates healthy, productive computation.
2. **Bad Speculation (< 10% Target)**:
   - Slots wasted because the CPU branch predictor guessed wrong on a conditional branch (`if/else` , dynamic dispatch). The pipeline executed instructions that had to be flushed.
   - **Optimization**: Branchless programming, lookup tables, compiler `-O3` profile-guided optimization (PGO).
3. **Frontend Bound (< 15% Target)**:
   - The CPU execution units were starved because the instruction fetch and decode units could not supply μops fast enough.
   - **Causes**: Instruction cache (ICache) misses, complex macro-instruction decoding stalls.
4. **Backend Bound**:
   - The CPU has fetched and decoded instructions, but execution units cannot proceed because resources are stalled.
   - Decomposes into:
     - **Memory Bound**: Waiting on L1/L2/L3 cache misses, DRAM latency, or cross-socket NUMA remote memory access.
     - **Core Bound**: Execution ports are saturated, divider/square-root units are busy, or vector execution units (FMA) are congested.

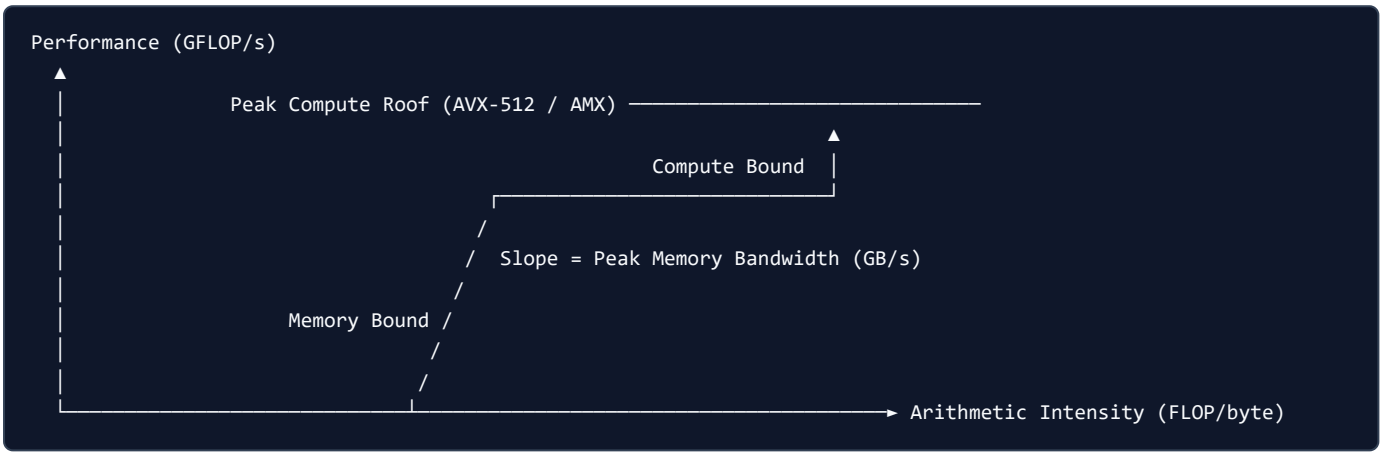
MODULE 6: Intel Advisor & The Roofline Model

6.1 The Roofline Model Formulation

The Roofline Model provides an intuitive 2D visual representation of a program's performance relative to the theoretical hardware limits of the target system:

- **X-axis**: Arithmetic Intensity (AI) measured in FLOPs per Byte of DRAM traffic.
- **Y-axis**: Performance measured in GigaFLOPs/sec or TeraFLOPs/sec.

$$\text{Attainable Performance} = \min\left(\text{Peak Performance}, \text{Arithmetic Intensity} \times \text{Peak Memory Bandwidth}\right)$$



6.2 Interpreting Roofline Plots

1. **Memory-Bound Ceiling (Left of the Knee)**:
   - Low Arithmetic Intensity ($< 5\ \text{FLOP/byte}$).
   - The CPU is starved for data. Adding faster compute cores will NOT speed up execution.
   - **Optimizations**: Cache tiling/blocking, operator fusion (fusing ReLU into Conv2D), data alignment, packing weights into contiguous memory.
2. **Compute-Bound Ceiling (Right of the Knee)**:
   - High Arithmetic Intensity ($> 20\ \text{FLOP/byte}$).
   - The application provides sufficient math operations per byte loaded from RAM.
   - **Optimizations**: Vectorization (AVX-512), mixed-precision computing (BF16 / INT8 utilizing Intel AMX), multi-threading.

MODULE 7: MPI Interconnect Profiling with ITAC

7.1 Intel Trace Analyzer and Collector (ITAC)

ITAC profiles distributed MPI applications by intercepting MPI calls (`MPI_Send`, `MPI_Recv`, `MPI_Allreduce`, `MPI_Barrier`) via runtime library preloading (`libVT.so`).

7.2 Common Distributed HPC Anti-Patterns

1. **Late Sender / Late Receiver Stalls**:
   - One compute node takes longer on its forward/backward pass (due to straggling threads, thermal throttling, or uneven data splits).

- All other $(P-1)$ nodes sit idle at the `MPI_Allreduce` barrier waiting for the straggler, resulting in wasted CPU cycles.

2. **Serialization Bottlenecks**:
   - Point-to-point communications organized in sequential chains ($0 \to 1 \to 2 \to 3$) rather than parallel trees.
3. **Imbalanced Partitions**:
   - Worker 0 receives 15,000 samples while Worker 3 receives 10,000 samples.

MODULE 8: Real-World Case Study: 4-Node Scaling Optimization

8.1 The Baseline Problem

A team deploys ResNet-50 training on a 4-node dual-socket Intel Xeon cluster (128 total cores) using standard PyTorch multi-processing over 1GbE Ethernet:

- 1 Node Throughput: 120 images/sec
- 4 Node Throughput: 165 images/sec
- **Scaling Efficiency**: $\frac{165}{4 \times 120} = 34.3\%$ (Extremely Poor!)

8.2 Systematic Diagnostic & Remediation

1. **Step 1 (VTune Profile)**: VTune reveals Memory Bound (62% pipeline slots stalled on DRAM).
   - *Fix*: Applied PyTorch Channels-Last memory format (`memory_format=torch.channels_last`) to exploit contiguous AVX-512 SIMD vectorization.
2. **Step 2 (Network Profiling)**: ITAC shows 58% of execution time spent in `MPI_Allreduce` over standard Ethernet TCP sockets.
   - *Fix*: Enabled InfiniBand RoCE RDMA via Intel oneCCL (`export I_MPI_FABRICS=shm:ofi`).
3. **Step 3 (Horovod Optimization)**:
   - Enabled Tensor Fusion Ring Buffer (`export HOROVOD_FUSION_THRESHOLD=67108864`).
   - Bound processes to CPU NUMA sockets using `numactl --cpunodebind` .

8.3 Post-Optimization Results

- 4 Node Optimized Throughput: 442 images/sec
- **Scaling Efficiency**: $\frac{442}{4 \times 120} = 92.1\%$!
- **Speedup Factor**: $2.68\times$ faster than the baseline cluster configuration.

🔬 SELF-ASSESSMENT QUESTIONS

1. Explain why Amdahl's Law enforces an asymptotic speedup limit on strong scaling while Gustafson's Law allows unbounded weak scaling.
2. Why does Ring-AllReduce transfer a constant volume of data per node regardless of the number of nodes in the cluster?
3. In Intel VTune's TMAM tree, what is the key difference between Backend Core Bound and Backend Memory Bound stalls?

4. When an application sits to the left of the knee on an Intel Advisor Roofline plot, why does increasing CPU clock frequency yield zero performance improvement?

GLA University HPC Masterclass | Module 4 Study Guide Completed